

# PCA

## Principal Component Analysis

---

Analítica de Datos



## Caso: Análisis de vinos con 13 propiedades químicas

- 13 features: acidez, pH, alcohol, azúcar, sulfatos, etc.

## Caso: Análisis de vinos con 13 propiedades químicas

- 13 features: acidez, pH, alcohol, azúcar, sulfatos, etc.
- **Problemas:**
  - Imposible de visualizar (no podemos graficar 13 dimensiones)
  - Muchas variables correlacionadas (acidez con pH, alcohol con densidad)
  - Costo computacional alto (entrenar modelos es más lento)
  - Riesgo de overfitting (muchas features para pocas observaciones)

## Caso: Análisis de vinos con 13 propiedades químicas

- 13 features: acidez, pH, alcohol, azúcar, sulfatos, etc.
- **Problemas:**
  - Imposible de visualizar (no podemos graficar 13 dimensiones)
  - Muchas variables correlacionadas (acidez con pH, alcohol con densidad)
  - Costo computacional alto (entrenar modelos es más lento)
  - Riesgo de overfitting (muchas features para pocas observaciones)
- **Pregunta:** ¿Podemos reducir features sin perder información?

# ¿Qué es PCA?

- **Principal Component Analysis:** Técnica de reducción de dimensionalidad
- Transforma features correlacionadas en componentes no correlacionados
- Ordena componentes por la varianza que explican
- Nos quedamos con los componentes más importantes
- **Resultado:** Menos features, misma información (casi)

## Ejemplo: Dataset de vinos

- Features originales: acidez, pH, sulfatos, cloruros
- Muchas están correlacionadas (acidez alta  $\rightarrow$  pH bajo)
- PCA crea nuevas variables (componentes principales):
  - **PC1:** Captura “composición ácida general” (máxima variabilidad)
  - **PC2:** Captura “balance químico” (segunda mayor variabilidad)
- En lugar de 13 features, podemos usar 2-3 componentes principales
- **Resultado:** Capturamos el 90 % de la información con menos variables

# ¿Por Qué Usar PCA?

## Motivaciones principales:

1. **Visualización:** Reducir a 2D o 3D para graficar
2. **Performance:** Menos features = entrenamiento más rápido
3. **Preprocesamiento:** Eliminar multicolinealidad
4. **Eliminación de ruido:** Componentes con poca varianza son ruido
5. **Compresión:** Almacenar datos de forma eficiente

# ¿Cómo Funciona PCA?

## Proceso simplificado:

1. **Estandarizar:** Todas las features con media 0, varianza 1

$$z = \frac{x - \mu}{\sigma}$$



# ¿Cómo Funciona PCA?

## Proceso simplificado:

1. **Estandarizar:** Todas las features con media 0, varianza 1

$$z = \frac{x - \mu}{\sigma}$$

2. **Calcular matriz de covarianza:** Mide cómo varían las features juntas

# ¿Cómo Funciona PCA?

## Proceso simplificado:

1. **Estandarizar:** Todas las features con media 0, varianza 1

$$z = \frac{x - \mu}{\sigma}$$

2. **Calcular matriz de covarianza:** Mide cómo varían las features juntas
3. **Eigenvectors y eigenvalues:** Direcciones de máxima varianza

# ¿Cómo Funciona PCA?

## Proceso simplificado:

1. **Estandarizar:** Todas las features con media 0, varianza 1

$$z = \frac{x - \mu}{\sigma}$$

2. **Calcular matriz de covarianza:** Mide cómo varían las features juntas
3. **Eigenvectors y eigenvalues:** Direcciones de máxima varianza
4. **Proyectar datos:** Transformar a los nuevos ejes

# ¿Cómo Funciona PCA?

## Proceso simplificado:

1. **Estandarizar:** Todas las features con media 0, varianza 1

$$z = \frac{x - \mu}{\sigma}$$

2. **Calcular matriz de covarianza:** Mide cómo varían las features juntas
3. **Eigenvectors y eigenvalues:** Direcciones de máxima varianza
4. **Proyectar datos:** Transformar a los nuevos ejes
5. **Seleccionar K componentes:** Quedarse con los más importantes

# Componentes Principales

- Cada componente es una **combinación lineal** de las features originales:

$$PC_1 = w_{1,1} \cdot x_1 + w_{1,2} \cdot x_2 + \dots + w_{1,p} \cdot x_p$$

# Componentes Principales

- Cada componente es una **combinación lineal** de las features originales:

$$PC_1 = w_{1,1} \cdot x_1 + w_{1,2} \cdot x_2 + \dots + w_{1,p} \cdot x_p$$

- Los componentes son **ortogonales** (no correlacionados)

# Componentes Principales

- Cada componente es una **combinación lineal** de las features originales:

$$PC_1 = w_{1,1} \cdot x_1 + w_{1,2} \cdot x_2 + \dots + w_{1,p} \cdot x_p$$

- Los componentes son **ortogonales** (no correlacionados)
- **PC1** captura la mayor varianza

# Componentes Principales

- Cada componente es una **combinación lineal** de las features originales:

$$PC_1 = w_{1,1} \cdot x_1 + w_{1,2} \cdot x_2 + \dots + w_{1,p} \cdot x_p$$

- Los componentes son **ortogonales** (no correlacionados)
- **PC1** captura la mayor varianza
- **PC2** captura la segunda mayor varianza (perpendicular a PC1)



# Componentes Principales

- Cada componente es una **combinación lineal** de las features originales:

$$PC_1 = w_{1,1} \cdot x_1 + w_{1,2} \cdot x_2 + \dots + w_{1,p} \cdot x_p$$

- Los componentes son **ortogonales** (no correlacionados)
- **PC1** captura la mayor varianza
- **PC2** captura la segunda mayor varianza (perpendicular a PC1)
- Y así sucesivamente...

## ¿Con cuántos componentes nos quedamos?

- Cada eigenvalue indica cuánta varianza captura ese componente

## ¿Con cuántos componentes nos quedamos?

- Cada eigenvalue indica cuánta varianza captura ese componente
- Proporción de varianza explicada:

$$\text{Varianza}_i = \frac{\lambda_i}{\sum_{j=1}^p \lambda_j}$$

## ¿Con cuántos componentes nos quedamos?

- Cada eigenvalue indica cuánta varianza captura ese componente
- Proporción de varianza explicada:

$$\text{Varianza}_i = \frac{\lambda_i}{\sum_{j=1}^p \lambda_j}$$

- **Regla práctica:** Quedarse con componentes que expliquen 80-95 % de la varianza

## ¿Con cuántos componentes nos quedamos?

- Cada eigenvalue indica cuánta varianza captura ese componente
- Proporción de varianza explicada:

$$\text{Varianza}_i = \frac{\lambda_i}{\sum_{j=1}^p \lambda_j}$$

- **Regla práctica:** Quedarse con componentes que expliquen 80-95 % de la varianza
- Graficar varianza acumulada para visualizar el trade-off

# Ejemplo de Varianza Explicada

## Caso: 64 features originales

- PC1 explica 50 % de la varianza

# Ejemplo de Varianza Explicada

## Caso: 64 features originales

- PC1 explica 50 % de la varianza
- PC2 explica 25 %

## Caso: 64 features originales

- PC1 explica 50 % de la varianza
- PC2 explica 25 %
- PC3 explica 15 %



## Caso: 64 features originales

- PC1 explica 50 % de la varianza
- PC2 explica 25 %
- PC3 explica 15 %
- PC4-PC64 explican menos del 10 %

## Caso: 64 features originales

- PC1 explica 50 % de la varianza
- PC2 explica 25 %
- PC3 explica 15 %
- PC4-PC64 explican menos del 10 %
- **Conclusión:** Con 3 componentes cubrimos 90 % de la varianza

## Caso: 64 features originales

- PC1 explica 50 % de la varianza
- PC2 explica 25 %
- PC3 explica 15 %
- PC4-PC64 explican menos del 10 %
- **Conclusión:** Con 3 componentes cubrimos 90 % de la varianza
- Reducimos de 64 features a 3 componentes!

# Implementación: PCA Básico

```
1 from sklearn.decomposition import PCA
2 from sklearn.preprocessing import StandardScaler
3
4 # Paso 1: Estandarizar
5 scaler = StandardScaler()
6 X_scaled = scaler.fit_transform(X)
7
8 # Paso 2: Aplicar PCA
9 pca = PCA()
10 X_pca = pca.fit_transform(X_scaled)
11
12 # Ver varianza explicada
13 print(pca.explained_variance_ratio_)
14 print(np.cumsum(pca.explained_variance_ratio_))
```

# Implementación: Reducir a K Componentes

```
1 # Reducir a 2 componentes para visualizacion
2 pca = PCA(n_components=2)
3 X_pca = pca.fit_transform(X_scaled)
4
5 print(f"Forma original: {X_scaled.shape}")
6 print(f"Forma reducida: {X_pca.shape}")
7
8 # Varianza total explicada
9 var_total = pca.explained_variance_ratio_.sum()
10 print(f"Varianza explicada: {var_total:.2%}")
```

# Visualización: Gráfico de Varianza

```
1 # Grafico de varianza acumulada
2 varianza_acum = np.cumsum(pca.explained_variance_ratio_)
3
4 plt.figure(figsize=(10, 6))
5 plt.plot(range(1, len(varianza_acum) + 1),
6          varianza_acum, marker='o')
7 plt.axhline(y=0.9, color='r', linestyle='--',
8             label='90% varianza')
9 plt.xlabel('Numero de Componentes')
10 plt.ylabel('Varianza Acumulada')
11 plt.legend()
12 plt.grid(True)
13 plt.show()
```

# Interpretación de Componentes

- Los componentes son difíciles de interpretar (combinaciones de features)
- Podemos ver qué features contribuyen más a cada componente
- Los “loadings” (pesos) nos indican la contribución:

```
1 # Ver loadings del primer componente
2 loadings = pca.components_[0]
3
4 # Features que mas contribuyen a PC1
5 top_features = np.argsort(np.abs(loadings))[-5:]
6 print(f"Top 5 features en PC1: {top_features}")
```

# Reconstrucción de Datos

- Podemos reconstruir una aproximación de los datos originales
- Si  $K < \text{número de features}$ , habrá pérdida de información
- Útil para compresión de datos

```
1 # Reconstruir datos originales
2 X_reconstructed = pca.inverse_transform(X_pca)
3
4 # Error de reconstruccion
5 mse = np.mean((X_scaled - X_reconstructed) ** 2)
6 print(f"Error de reconstruccion (MSE): {mse:.4f}")
```



## PCA como preprocesamiento:

- Puede mejorar el performance de modelos supervisados
- Reduce tiempo de entrenamiento
- Mitiga overfitting (menos features)
- Elimina multicolinealidad

# Implementación: Pipeline con PCA

```
1 from sklearn.pipeline import Pipeline
2 from sklearn.linear_model import LogisticRegression
3
4 # Pipeline: Scaler -> PCA -> Modelo
5 pipeline = Pipeline([
6     ('scaler', StandardScaler()),
7     ('pca', PCA(n_components=10)),
8     ('classifier', LogisticRegression())
9 ])
10
11 # Entrenar
12 pipeline.fit(X_train, y_train)
13
14 # Evaluar
15 score = pipeline.score(X_test, y_test)
16 print(f"Accuracy con PCA: {score:.3f}")
```

# Ventajas de PCA

- **Reduce dimensionalidad** automáticamente
- **Elimina multicolinealidad:** Componentes ortogonales
- **Visualización:** Permite graficar datos en 2D/3D
- **Eficiencia:** Reduce costo computacional
- **Elimina ruido:** Descarta componentes con poca varianza

# Limitaciones de PCA

- **Pérdida de interpretabilidad:** Componentes son combinaciones de features
- **Asume linealidad:** Solo captura relaciones lineales
- **Sensible a outliers:** Valores extremos distorsionan componentes
- **Requiere estandarización:** SIEMPRE normalizar primero
- **No supervisa:** No considera la variable objetivo

# ¿Cuándo Usar PCA?

## Casos ideales:

- Muchas features correlacionadas
- Visualizar datos de alta dimensión
- Entrenamiento muy lento por cantidad de features
- Problemas de multicolinealidad
- Compresión de datos (imágenes, audio)

## ¿Cuándo NO Usar PCA?

- Features no correlacionadas (PCA no reduce mucho)
- Interpretabilidad es crítica
- Relaciones no lineales (usar kernel PCA o autoencoders)
- Muy pocas features (no tiene sentido)

## PCA:

- Lineal, rápido
- Bueno para ML

## t-SNE:

- No lineal
- Solo visualización

## Feature Selection:

- Más interpretable
- No combina features

## Autoencoders:

- Redes neuronales
- Captura no linealidad

## Consideraciones Importantes

- **SIEMPRE estandarizar** antes de PCA
- Elegir K basado en varianza acumulada (80-95 %)
- Trade-off: menos componentes = menos info pero más eficiencia
- Comparar performance con y sin PCA
- Usar gráfico de varianza acumulada para decidir K



**¿Dudas?**  
**¿Consultas?**



Universidad de  
**SanAndrés**